



IDENTIFY THE INHERITANCE

- A bird can fly.
- A machine can be automated.
- Now imagine a drone.
- A drone is both “flyable” and “controllable.”

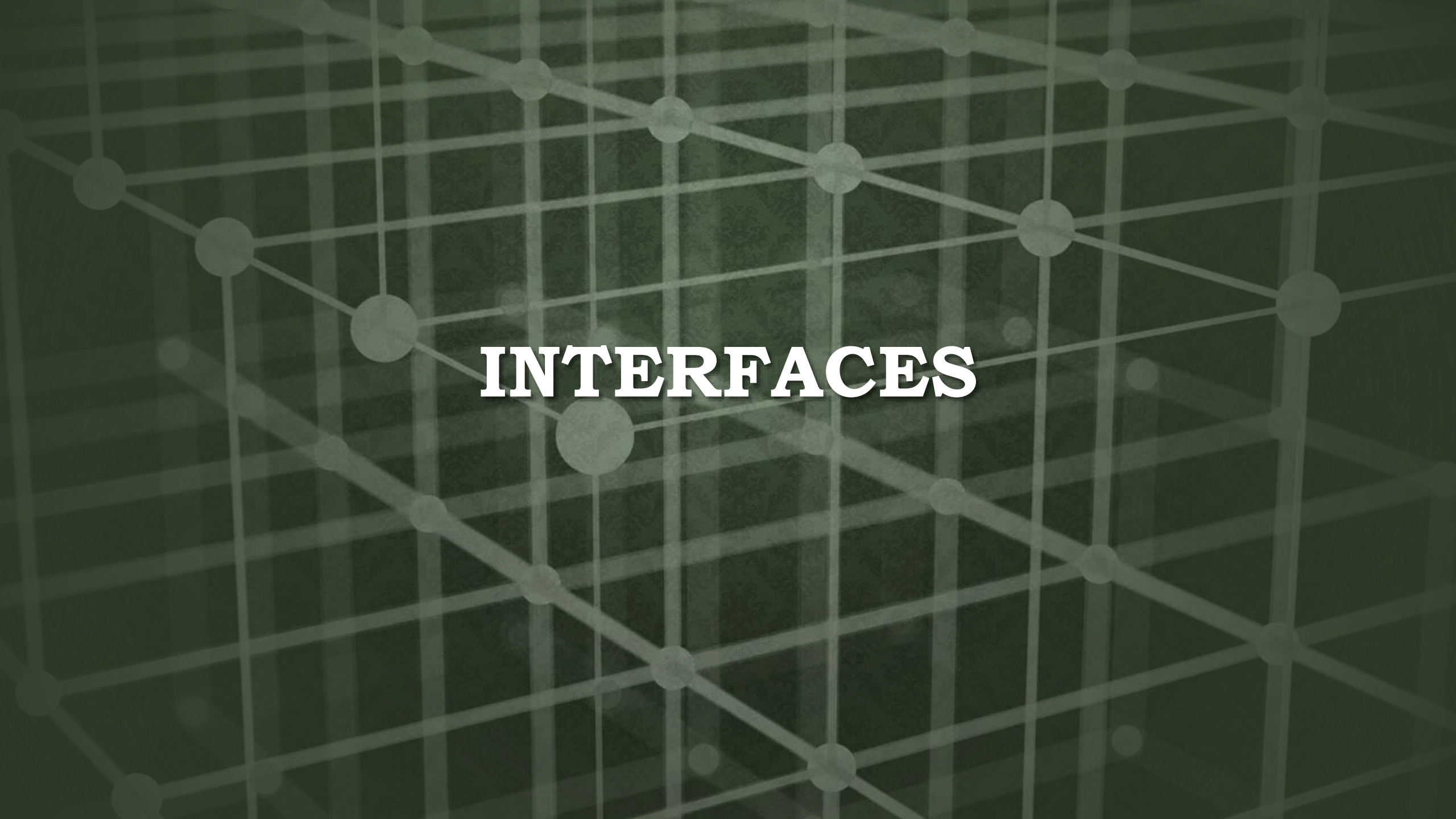
**YES, IT IS
MULTIPLE
INHERITANCE**

THE PROBLEM

Java, a class cannot inherit from two classes at the same time.

Inheritance allows only one parent class.

But sometimes an object should have multiple capabilities.



INTERFACES

```
interface Flyable {  
    void fly();  
}  
  
interface Controllable {  
    void control();  
}  
  
class Drone implements Flyable, Controllable {  
  
    public void fly() {  
        System.out.println("Drone is flying in the sky.");  
    }  
  
    public void control() {  
        System.out.println("Drone is being controlled remotely.");  
    }  
}
```

EXAMPLE

**WHY IMPLEMENT
INSTEAD OF
EXTEND?**

EXPLANATION



Interfaces should not be viewed as normal classes.



A class contains:

Variables

Constructors

Fully defined methods

Objects can be created from it



An interface mainly contains method declarations (a contract) that other classes must follow.

EXPLANATION



For example, if Flyable says a class must have a fly() method,



It does not tell how flying should happen. Each class decides that itself.



So an interface is more like: “Any class that uses me must provide these methods.”



That is why we do not use inheritance words like “is-a parent class” in the same way.



That is why Java uses the implements keyword.

IN A NUTSHELL

extends = getting
existing features

implements =
agreeing to follow
rules or a contract

```
interface Flyable {  
    void fly();  
}  
  
interface Controllable {  
    void control();  
}  
  
class Drone implements Flyable, Controllable {  
  
    public void fly() {  
        System.out.println("Drone is flying in the sky.");  
    }  
  
    public void control() {  
        System.out.println("Drone is being controlled remotely.");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Drone d = new Drone();  
  
        d.fly();  
        d.control();  
    }  
}
```

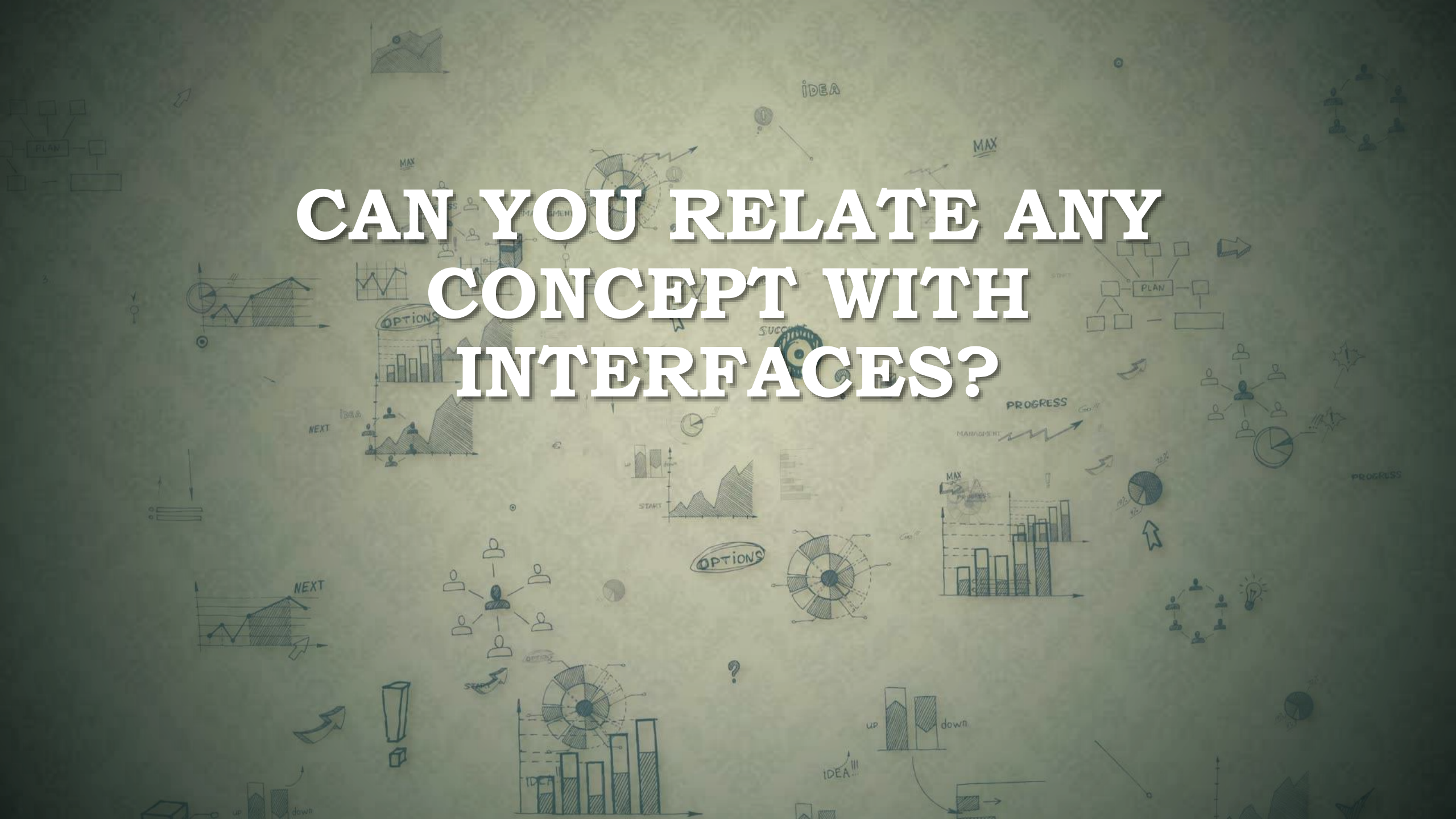
**FULL
WORKING
CODE**

**IMPLEMENTATION
METHODS MUST
BE PUBLIC IN
CHILD CLASSES**

**Interface methods are
public abstract by default.**

**Implementing class must
use public.**

**This ensures the contract of
the interface is respected.**



**CAN YOU RELATE ANY
CONCEPT WITH
INTERFACES?**

INTERFACES



Another way to implement abstraction.



An abstract class can provide:

Abstract methods
Normal methods with implementation
Variables



But an interface provides only the rules that a class must follow.

INTERFACES

An **interface** is a collection of abstract methods that a class must implement.

Interfaces **do not have constructors**.

No method body (in traditional Java).

Variables are public static final by default.

Purpose: To define a contract for classes.

IN A NUTSHELL



Abstract class = partial
abstraction



Interface = complete
abstraction (in traditional Java)

WHY USE INTERFACES?



SOLVE THE **MULTIPLE INHERITANCE PROBLEM** IN JAVA.



ALLOW **DIFFERENT CLASSES** TO IMPLEMENT THE SAME BEHAVIOR IN THEIR OWN WAY.



ENABLE **POLYMORPHISM** THROUGH A COMMON INTERFACE REFERENCE.



IMPLEMENT COMPLETE ABSTRACTION.

INTERFACE VS ABSTRACT CLASS

Feature	Abstract Class	Interface
Methods	Can be abstract or concrete	Abstract by default (Java 8+: can have default/static)
Variables	Can have instance variables	public static final by default
Inheritance	extends only one class	implements multiple interfaces
Constructor	Yes	No
Use	Partial abstraction	Complete abstraction / contract

VARIABLES IN INTERFACE

- MAX_SPEED is a constant — you **cannot change it**.
- Interface variables can be accessed **directly using the interface name**.
- Any class implementing the interface can **use this constant**.

```
// Interface with a variable
interface Vehicle {
    int MAX_SPEED = 120; // automatically public static final
    void drive();
}

// Implementing class
class Car implements Vehicle {
    public void drive() {
        System.out.println("Car is driving at max speed " + MAX_SPEED + " km/h");
    }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car();
        myCar.drive();

        // Accessing interface variable directly
        System.out.println("Maximum allowed speed: " + Vehicle.MAX_SPEED);
    }
}
```

```
// Interface with default and static methods
interface Flyable {
    void fly(); // abstract method

    default void land() {        // default method
        System.out.println("Landing...");
    }

    static void info() {         // static method
        System.out.println("All flyables can fly");
    }
}

// Class implementing Flyable
class Drone implements Flyable {
    public void fly() {          // implement abstract method
        System.out.println("Drone is flying");
    }
}

public class Main {
    public static void main(String[] args) {
        Flyable.info();         // call static method

        Flyable d = new Drone();
        d.fly();                 // call implemented method
        d.land();                // call default method
    }
}
```

MODERN JAVA: DEFAULT & STATIC METHODS

- fly() → must be implemented (abstract).
- land() → default, optional to override.
- info() → static, called using interface name.

POLYMORPHISM WITH INTERFACE

```
// Interface
interface Animal {
    void sound(); // abstract method
}

// Implementing class
class Cat implements Animal {
    public void sound() { // must be public
        System.out.println("Meow");
    }
}

public class Main {
    public static void main(String[] args) {
        // Interface reference pointing to implementing class object
        Animal a = new Cat();
        a.sound(); // Output: Meow
    }
}
```

WHAT WILL BE OUTPUT OF THE FOLLOWING CODE?

```
interface Flyable {
    void fly();
}

interface Controllable {
    void control();
}

class Drone implements Flyable, Controllable {
    public void fly() {
        System.out.println("Drone is flying in the sky.");
    }

    public void control() {
        System.out.println("Drone is being controlled remotely.");
    }
}

public class Main {
    public static void main(String[] args) {
        Flyable obj = new Drone();
        obj.fly();
        obj.control();
    }
}
```

EXPLANATION



obj is declared as type **Flyable**, so the **compiler only knows Flyable methods are available.**



Flyable interface **does not have a control() method.**



That's why `obj.control();` will give a **compile-time error.**



How to fix it:

Option 1 – Use the class type

Option 2 – Use both interfaces

```
interface PaymentMethod {  
    void pay(double amount);  
}  
  
class CreditCard implements PaymentMethod {  
    public void pay(double amount) {  
        System.out.println("Paid $" + amount + " using Credit Card.");  
    }  
}  
  
class PayPal implements PaymentMethod {  
    public void pay(double amount) {  
        System.out.println("Paid $" + amount + " using PayPal.");  
    }  
}  
  
public class MainPayment {  
    public static void main(String[] args) {  
        PaymentMethod card = new CreditCard();  
        PaymentMethod paypal = new PayPal();  
        card.pay(100.0);  
        paypal.pay(50.0);  
    }  
}
```

APPLICATION 01 – PAYMENT SYSTEMS

```
interface Notifier {  
    void send(String message);  
}  
  
class EmailNotifier implements Notifier {  
    public void send(String message) {  
        System.out.println("Email sent: " + message);  
    }  
}  
  
class SMSNotifier implements Notifier {  
    public void send(String message) {  
        System.out.println("SMS sent: " + message);  
    }  
}  
  
public class MainNotifier {  
    public static void main(String[] args) {  
        Notifier email = new EmailNotifier();  
        Notifier sms = new SMSNotifier();  
        email.send("Welcome!");  
        sms.send("Your OTP is 1234");  
    }  
}
```

APPLICATION 02 – NOTIFICATIONS

```
interface Login {
    boolean authenticate(String username, String password);
}

class AdminLogin implements Login {
    public boolean authenticate(String username, String password) {
        return username.equals("admin") && password.equals("1234");
    }
}

class UserLogin implements Login {
    public boolean authenticate(String username, String password) {
        return username.equals("user") && password.equals("abcd");
    }
}

public class MainLogin {
    public static void main(String[] args) {
        Login admin = new AdminLogin();
        Login user = new UserLogin();
        System.out.println("Admin login: " +
admin.authenticate("admin", "1234"));
        System.out.println("User login: " + user.authenticate("user",
"abcd"));
    }
}
```

APPLICATION 03 – LOGIN SYSTEMS

```
interface Walkable {
    void walk();
}

interface Speakable {
    void speak();
}

interface Rechargeable {
    void recharge();
}

class HumanoidRobot implements Walkable, Speakable, Rechargeable {
    public void walk() {
        System.out.println("Humanoid walking");
    }

    public void speak() {
        System.out.println("Humanoid speaking");
    }

    public void recharge() {
        System.out.println("Humanoid recharging");
    }
}

public class MainRobot {
    public static void main(String[] args) {
        HumanoidRobot robot = new HumanoidRobot();
        robot.walk();
        robot.speak();
        robot.recharge();
    }
}
```

APPLICATION 04 – ROBOTICS

SUMMARY

✓ Interfaces = pure abstraction

🔍 Use implements keyword

📱 Can implement **multiple interfaces**

🧬 Enables **polymorphism and code reusability**

💻 Modern Java: default and static methods